

AegisMed: Healthcare DevSecOps XAI Framework

Complete Faculty Defense & Technical Examination Master Guide

Easy Explanation | Deep Mathematical Formulation | UI Walkthrough | Backend Code | Viva Q&A

1. The Working in an Easy Manner (High-Level Intuition)

1.1 The Hospital Problem: Why Standard Security Tools Fail

Imagine an airport security guard who checks luggage for knives and liquids. That guard does a great job at the airport. Now take that same guard and put them in a hospital operating room. A surgeon walks in holding a scalpel. The security guard screams, tackles the surgeon, and shuts down the hospital because 'a sharp blade was detected!' Meanwhile, a nurse leaves an unencrypted clipboard containing 5,000 cancer patient records on a public park bench outside, and the guard completely ignores it because a clipboard is not a weapon.

This is exactly what happens with commercial software security tools like SonarQube, Snyk, and Checkmarx. They were built for generic e-commerce or social media websites. They do not know what a Social Security Number (SSN), Medical Record Number (MRN), or EHR vitals record is. They flood developers with 200 false warnings about harmless variables, while completely missing catastrophic legal violations like storing unencrypted patient vitals in a database or exposing patient data in application logs.

1.2 How AegisMed Works in 4 Simple Steps

- **Step 1 - Spotting the Patient Data:** When a developer submits code, AegisMed scans the code like a doctor. It looks for clinical variables (SSN, MRN, patient ID, diagnosis, vitals) using the HIPAA Safe Harbor 18 statutory list.
- **Step 2 - Following Data Flow Like Colored Ink:** AegisMed uses 'Taint Analysis'. Think of patient data as bright red ink. If a developer assigns $x = \text{ssn}$, the red ink spreads to x . If x is inserted into a database or printed in logs without encryption, AegisMed flags a critical HIPAA violation.
- **Step 3 - Consulting Real Federal Hack Records:** Assessing risk with arbitrary numbers, AegisMed consults 1,656 real hospital breach cases from the U.S. Department of Health and Human Services (HHS). Because the federal data proves that 46.6% of hospital breaches happen due to unencrypted data, our AI heavily penalizes unencrypted patient flows.
- **Step 4 - Finding the Minimum Fix Needed (MILP):** Instead of just saying 'Build Failed' and leave developers stranded. AegisMed acts like a GPS navigator: it solves a mathematical puzzle (MILP) to find the absolute minimum set of code fixes required to make the software safe and unblock deployment. Then it simulates the fix and re-checks the code to guarantee the gate opens.

2. Technical & Mathematical Foundations (Faculty Rigor)

2.1 AST Semantic Taint-Flow Analysis (ast.NodeVisitor)

Source code is compiled into an Abstract Syntax Tree $T = (V, E)$ using Python's native ast module. The analyzer traverses nodes to track dataflow:

- **HIPAA Safe Harbor 18 Ontology:** Identifies ePHI parameters and variables across clinical naming conventions: $S_{\text{ePHI}} = \{\text{ssn}, \text{mrn}, \text{patient_id}, \text{diagnosis}, \text{vitals}, \text{dob}, \text{encounter}, \text{Observation}, \text{Patient}, \text{Condition}\}$.
- **Variable Aliasing Propagation:** Monitors ast.Assign nodes. If right-hand side is in tainted_vars, the left-hand symbol inherits taint (e.g., $\text{alias} = \text{ssn}$ implies alias in T_{tainted}).
- **Async Route Inspection:** Overrides visit_AsyncFunctionDef alongside visit_FunctionDef. Inspects FastAPI/SMART-on-FHIR route decorators to trap wildcard permission scopes (*.*).
- **Multi-Syntax SQLi Sinks:** Detects SQL queries built via f-strings (f'SELECT...{ssn}'), .format() invocations, and modulo (% ssn) formatting.
- **Strict Sanitizer Verification:** Validates that the tainted identifier is passed as an actual input argument to an authorized cipher (e.g., AESGCM.encrypt, kms.generate_data_key, hashlib.sha256). Superficial or dummy functions (e.g., rehash_noop('dummy')) are rejected.

2.2 Monotonic Gradient Boosting Risk Engine & HHS Priors

A 10-dimensional feature vector x is extracted representing SAST criticals, SAST highs, DAST endpoints, SCA CVSS, vulnerable packages, PHI leak score, unencrypted data flows, IaC misconfigurations, compliance penalty, and domain risk weight.

- **Empirical Federal Priors:** At initialization, `_load_hhs_priors()` parses datasets/hhs_major_data_breaches.csv (1,656 incidents): Theft/Loss of unencrypted data: 46.6%, Unauthorized disclosures: 26.0%, Hacking/IT: 13.2%.
- **CISA KEV Multiplier:** Vulnerabilities in the CISA Known Exploited Vulnerabilities feed (e.g., CVE-2023-4863 libwebp buffer

- **Monotonic Constraints:**
- overflow) receive a 1.15x CVSS exploit multiplier.

Trained via HistGradientBoostingRegressor(monotonic_cst=[1, 1, 1, 1, 1, 1, 1, 1, 0]). Guarantees $df(x)/dx_i \geq 0$, proving mathematically that fixing a vulnerability can never counterintuitively increase risk. 5-fold CV yields $R^2 = 0.962$.

2.3 Two-Tier Cooperative Game-Theoretic Explainability (Tree-SHAP & Finding Shapley)

AegisMed implements two distinct layers of cooperative game theory to ensure complete transparency and auditability:

- Tier 1 - Feature-Level Tree-SHAP:** Uses official shap.TreeExplainer (Lundberg et al., Nature MI 2020) across all trees in the ensemble. Satisfies Local Efficiency: $f(x) = E[f] + \sum(\phi_i)$ with residual error $< 1e-14$.
- Tier 2 - Finding-Level Shapley Attribution:** Treats each detected vulnerability finding F_j as a player in a cooperative game. Characteristic function $v(S)$ evaluates the risk model over subset S . Computes exact Shapley blame Φ_j :

$$\Phi_j = \sum_{S \subset F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} * [v(S \cup \{j\}) - v(S)]$$

Satisfies Local Efficiency: $\sum(\Phi_j) = v(All) - v(Empty)$ with zero residual. Blame is aggregated directly into statutory clauses: HIPAA Section 164.312(a)(2)(iv) (Encryption at Rest), Section 164.312(b) (Audit Controls), Section 164.312(e)(1) (Transmission Security), and FDA SaMD 524B.

2.4 Discrete MILP Counterfactual Optimizer & Closed-Loop Re-Scan

When a deployment is BLOCKED, the system solves a 0-1 Mixed-Integer Linear Program via scipy.optimize.milp:

$$\min_{\{z\}} \sum_j (c_j * z_j) \quad \text{subject to} \quad \sum_j (\Phi_j * z_j) \geq \Delta_{risk} \quad \text{and} \quad z_j \in \{0, 1\}$$

where c_j is remediation effort in points, and hard bounds enforce $z_j = 1$ for all mandatory HIPAA critical violations. Upon solving z^* , the system synthesizes cryptographic patches, updates in-memory code, and performs an automated closed-loop AST re-scan. Only when the re-scan confirms zero remaining critical flows does the gate transition to APPROVED_AUTO_DEPLOY.

2.5 Empirical Benchmark Evaluation

Formal benchmark comparison on clinical test cases (variable aliasing, async FHIR, format SQLi, and envelope encryption):

Analysis Engine	Prec	Recall	F1	FP	FN	Latency
Generic Regex Matching	0.800	0.667	0.727	1	2	0.4 ms
Generic AST Checker (Bandit)	0.750	0.500	0.600	1	3	0.5 ms
AegisMed AST Taint-Flow (Ours)	1.000	1.000	1.000	0	0	0.5 ms

3. Webpage Features & Component Walkthrough

The AegisMed user interface is structured as a minimalist, dark slate engineering dashboard (Linear/Vercel aesthetic) organized into 5 functional tabs and high-level control headers:

3.1 App Header & Top Navigation Controls

- Target Scenario Selector:** Dropdown to switch between 4 healthcare scenarios: EHR Patient Portal (High Risk), SMART-on-FHIR Gateway (Async Scope/Log Leak), SaMD Imaging Worker (FDA 510k/LibWebP), and Hardened Hospital API (Clean Baseline).
- Training Datasets Button:** Opens a modal detailing the 5 physical datasets (1,656 HHS breach records, NVD CVE feeds, and 2,500 training build matrix).
- AWS Architecture Button:** Opens modal comparing Phase 1 (Academic single EC2 container, <\$3 spend) vs Phase 2 (Enterprise Fargate + SageMaker + HealthLake).
- Export Audit Certificate:** Generates an official printable PDF audit certificate for hospital compliance sign-off.
- Run CI/CD Scan Button:** Triggers the full 6-stage DevSecOps assessment pipeline on the selected codebase.

3.2 Tab 1: Overview & Policy Gate

- CI/CD Pipeline Tracker:** Visualizes 5 stages: Code Commit -> AST Taint Ingestion -> Statutory Mapping -> Monotonic ML/SHAP -> Policy Gate Decision.
- Policy Gate Decision Banner:** Displays color-coded gate state: BLOCKED (red), MANUAL REVIEW (amber), or APPROVED (emerald) with specific action directives.
- 4 Key Metric Boxes:** Displays continuous AI Risk Score (0-100), Patient Breach Likelihood (%), Regulatory Penalty score, and Monotonic Model Verification R^2 (0.962).
- Top Root-Cause Drivers:** Lists the specific vulnerabilities causing the largest upward SHAP risk impact (e.g., Unencrypted Data Flows: +38.2 pts).
- Multi-Stakeholder Narratives:** Interactive toggle between Developer Technical Brief (file and line coordinates) and Auditor Compliance Brief (HIPAA statutory clauses).

3.3 Tab 2: AST Taint & Dataflow

- Visual Dataflow Tracing Bar:** Visual breadcrumb diagram: ePHI Source -> Variable Aliasing -> Strict Sanitizer Check -> Clinical Insecure Sink.

- **Discovered Security Findings:** Card list of every finding showing ID, Severity, File/Line, Description, and statutory regulatory clause.
- **Live Custom Code Scanner:** Interactive code editor allowing developers to paste custom Python code or Dockerfiles, inspect AST rules, and scan instantaneously.

3.4 Tab 3: Two-Tier Game-Theoretic XAI

- **Tier 1 Tree-SHAP Chart:** Minimalist horizontal bar chart showing marginal positive/negative feature contributions relative to base expected risk $E[f]$.
- **Tier 2 Finding-Level Shapley Table:** Detailed cooperative game attribution table displaying Finding ID, Type, Severity, Statutory Clause, Shapley Blame (Φ_j), and % Risk Impact.
- **Aggregated Clause Attribution:** Summary cards aggregating total points assigned to specific legal clauses (e.g., HIPAA Section 164.312(a)(2)(iv): +38.2 pts).

3.5 Tab 4: Discrete MILP & Remediations

- **Discrete MILP Optimization Plan:** Displays the optimal 0-1 decision vector z^* , developer effort points, projected score (13.5), and Closed-Loop AST Re-Scan Verification status.
- **Interactive What-If Simulator:** Sliders for Critical SAST, Unencrypted Flows, PHI Leak Risk, and Dependency CVSS with instant debounced re-calculation.
- **Cryptographic Code Diffs:** Side-by-side Before/After code patches with unified diff formatting (AWS KMS AES-256 envelope encryption, parameterized SQL, SMART-on-FHIR scopes).

3.6 Tab 5: Compliance & Empirical Evidence

- **Empirical Benchmark Table:** Formal comparison table proving AegisMed's 1.000 F1 score vs 0.727 (Regex) and 0.600 (Generic AST).
- **HHS OCR Breach Statistics:** Empirical breakdown of the 1,656 federal hospital records (Theft: 46.6%, Unauthorized Disclosure: 26.0%, Hacking: 13.2%).
- **Healthcare Statutory Matrix:** Comprehensive mapping of every technical safeguard in HIPAA 45 CFR Section 164.312, FDA SaMD 524B, and HL7 FHIR.

4. Backend Codebase Architecture (File-by-File)

The backend is implemented in Python 3.11+ using FastAPI, scikit-learn, official Tree-SHAP, and SciPy:

- **backend/main.py:** FastAPI application entry point. Serves API endpoints (`/api/scan`, `/api/xai/whatif`, `/api/scenarios`, `/api/datasets/info`, `/api/audit/export`) and mounts the static UI.
- **backend/scanner_simulator.py:** HealthcareASTTaintAnalyzer inheriting from `ast.NodeVisitor`. Parses Safe Harbor 18 ontology, propagates taint across variable aliases, inspects `visit_AsyncFunctionDef`, detects `format/% SQLi` sinks, and performs strict sanitizer verification. Also contains SCA dependency and IaC Dockerfile scanners.
- **backend/ml_engine.py:** MLRiskEngine class. Loads `datasets/hhs_major_data_breaches.csv` in `_load_hhs_priors()`. Trains `HistGradientBoostingRegressor` with monotonic constraints (`monotonic_cst=[1,1,1,1,1,1,1,1,0]`, $R^2=0.962$) and Random Forest Classifier for breach probability. Ingests CISA KEV multiplier.
- **backend/xai_engine.py:** XAIEngine class. Computes 10-feature Tree-SHAP via `shap.TreeExplainer`. Implements `compute_finding_level_shapley()` via coalition evaluation. Implements `solve_milp_counterfactual()` using `scipy.optimize.milp` with closed-loop AST re-scan verification.
- **backend/healthcare_compliance.py:** HealthcareComplianceEngine class. Evaluates findings against HIPAA Section 164.312 (Technical Safeguards), FDA SaMD Section 524B (SBOM & vulnerability mitigation), and HL7 FHIR least-privilege scopes.
- **backend/remediation_engine.py:** RemediationEngine class. Provides 9 verified cryptographic code patch templates with unified diffs, line replacements, and an optional LLM synthesis hook.

5. Faculty Viva Defense Q&A Cheat Sheet

Anticipated faculty evaluation questions and strategic high-scoring responses:

Q1: What is the core novelty of your project? Why is it patentable?

A: We introduce 4 genuine patentable novelties: (1) An AST semantic taint analyzer tracking ePHI from Safe Harbor 18 sources across variable aliases to sinks, with strict sanitizer argument verification; (2) An empirical ML risk model grounded on 1,656 federal HHS breach records enforcing non-decreasing monotonic constraints ($df/dx_i \geq 0$); (3) Two-tier cooperative game theory providing exact finding-level Shapley blame (Φ_j) aggregated to HIPAA clauses; and (4) A discrete 0-1 MILP optimizer solved via `scipy.optimize.milp` with closed-loop AST re-scan verification.

Q2: How does your tool differ from SonarQube or Snyk?

A: Generic tools are domain-blind. They do not recognize clinical ePHI identifiers (SSN, MRN, vitals), cannot detect taint propagation through variable aliases into unencrypted database sinks, and provide no cooperative game theory or MILP counterfactual optimization to tell developers the minimal effort required to unblock a deployment.

Q3: How do you prove your Shapley values are mathematically valid?

A: We satisfy the Local Efficiency Axiom at both tiers: for Tree-SHAP, $\sum(\phi_i) = f(x) - E[f]$ (residual $< 1e-14$); for finding-level Shapley, $\sum(\Phi_j) = v(\text{All}) - v(\text{Empty})$ (residual 0.000). Both are verified automatically in our 49-test pytest suite.

Q4: Why did you use Monotonic Constraints in your Machine Learning model?

A: In standard decision trees, fixing a security vulnerability can counterintuitively increase the predicted risk score due to unconstrained tree splitting. By configuring `HistGradientBoostingRegressor` with `monotonic_cst`, we mathematically guarantee that $df/dx_i \geq 0$ across all vulnerability features, achieving 5-fold CV $R^2 = 0.962$.

Q5: What is Closed-Loop Re-Scan Verification?

A: When the MILP optimizer selects code fixes (z^*), AegisMed does not rely on naive guesswork. It physically applies the cryptographic patches to the source code and re-executes the AST taint scanner. Only if the re-scan confirms that all critical flows have been eliminated and the recalculated score drops below 24.0 does the CI/CD gate transition to `APPROVED_AUTO_DEPLOY`.

Q6: How can this be deployed under a \$50 AWS Learner Lab budget?

A: The prototype is containerized as a single Docker container running FastAPI, scikit-learn, and Tree-SHAP. On an EC2 t2.micro or t3.micro instance, it costs ~\$0.0116/hour. Running the server for an entire week of demonstrations costs less than \$2.00, consuming under 4% of the \$50 student credit. Our documentation provides the full roadmap to Amazon SageMaker and AWS HealthLake for enterprise hospital networks.